Swagger / OpenAPI standardization for Jenkins REST API

Google Summer of Code Program 2025 Project Draft Proposal



Pratik Mane

Mail / GitHub

Table of Contents

Project Abstract 3

Project Description 3

Project Deliverables 5

High-level Overview of Architecture Diagram: 6

Implementation 7

Phase 1: New Plugin Construction & Locate Javadocs 7

Phase 2: Filtering Mechanism for REST API Extraction 8

Phase 3: Conversion to OpenAPI Specification 9

Phase 4: Versioning and Exposing Swagger UI for Jenkins Core 10

Phase 5: Versioning and Exposing Swagger UI for Plugins 11

Phase 6: Automation Integration with Maven HPI Plugin Skeleton 12

Proposed Timeline 12

Phase 0: Pre-Bonding Period (till 8th May) 12

Phase 1: Community Bond Begins (8th May - 1st June) 12

Phase 2: Coding Phase I (2nd June - 13th July) 12

Phase 3: Coding Phase II (14th July to 25th August) 12

Major Challenges Foreseen 13

Availability and Commitments 14

Continued Involvement 14

References 14

Relevant Background Experience 15

Personal 16

Open Source Contributions 17

Skill Sets 17

Reference Links and URLs 18

Project Abstract

This project aims to develop a Jenkins plugin to automatically extract and document REST API endpoints from both Jenkins core and plugins using **OpenAPI/Swagger** specifications. The solution will scan existing javadocs to identify endpoints, extract their metadata, generate standardized documentation, and expose versioned API documentation through dedicated URLs. Additionally, it will extend Jenkins plugin skeleton generator to facilitate REST API creation in new plugins.

Project Description

Jenkins (core & plugins) ecosystem currently has a well maintained documentation in terms of Remote Access API & Java Docs, but it lacks in-depth documentation and testing for expected response for appropriate request for REST APIs endpoints. This makes it **challenging for developers to understand and interact with its APIs**. To overcome all this swagger comes into picture, it can address these challenges by providing its key features such as :

- ➔ **API Documentation:** It generates accurate documentation for specified endpoints, their description, parameters with data types, return values, endpoints request type (GET, POST, UPDATE, DELETE, etc), response status, etc. Which helps to provide reliable reference material.

- ➔ **API Testing:** It supports testing API endpoints by sending requests and receiving responses directly within the interface. Which helps to understand the behavior of each endpoint.

- ➔ **Interactive UI:** Unables to visualize and interact with the API's resources without having any of the implementation logic in place. from the provided OpenAPI spec.

Swagger / OpenAPI specifications are written in **YAML/JSON** format, making them readable for both humans and machines. These formats are used to generate REST APIs Documentation according to the OpenAPI specification. To maintain the REST APIs in the OpenAPI Specification format for the Jenkins core and all its **2000+ plugins** is time-consuming. Makes it difficult to manually convert these REST APIs into an OpenAPI spec.

The goal of this project is to create a Jenkins plugin that automates the extraction, conversion, and generation of REST API documentation in the OpenAPI Specification format. By leveraging Swagger and associated tools, the plugin will streamline the documentation process, making it more efficient and for Jenkins core and plugin developers. This solution will reduce the manual effort involved in maintaining accurate and up-to-date API documentation and expose versioned API documentation separately for jenkins core (http://localhost/rest/api/1.0) & plugins (http://localhost/rest/plugin/api/1.0) through dedicated URLs. Additionally, it will extend Jenkins plugin skeleton generator to facilitate REST API creation in new plugins.

Why is the project important ?

Over the past couple of months, as I've immersed myself in the open-source journey with Jenkins, I've often used Java Docs to explore Jenkins features, but they frequently encounter limitations when it comes to real-time testing of requests and responses. They need to seek clarity on what an endpoint truly does in the context of REST APIs and desire the ability to test it seamlessly through tools like Postman. With this in mind, I believe that this tool will provide developers with the opportunity to test REST APIs while reading the Jenkins documentation, enhancing both their workflow and overall experience.

Why did I choose this project ?

To speak honestly, when I first came across this project idea, I wasn't aware of Swagger but was familiar with REST APIs. However, after attending the Quick start meeting by Oleg Nenashev & Kristin Whetstone and reviewing the sample documentation generated with the folder-auth plugin (it has unresolved security vulnerability now) by Abhyudaya Sharma, I became more acquainted with REST APIs and it piqued my interest in Swagger. Inspired by this, I implemented Swagger in my Flask Project (working locally) and was genuinely impressed by its capabilities. Now, as I focus on automation, I can see how integrating

Swagger will significantly enhance the API documentation for Jenkins, taking it to the next level.

Project Deliverables

I have subdivided the project deliverables into five phases as follows:

In Phase 1, I will create a new Jenkins plugin using the Maven HPI Plugin framework that mimics the functionality of the Jenkins Javadoc Plugin to locate and archive the generated HTML Javadocs.

In Phase 2, I will enhance your plugin with a filtering module that processes the archived HTML Javadocs to extract only the information related to REST APIs.

In Phase 3, I will create a mechanism which will convert the filtered REST API metadata info from Phase 2 to an OpenAPI (Swagger) specification (in JSON or YAML format) and store the resulting files separately for Jenkins core and plugins.

In Phase 4, I will implement a versioning mechanism and expose a dedicated endpoint (e.g., `http://localhost/rest/api/1.0`) to serve the OpenAPI spec for Jenkins core via Swagger UI.

In Phase 5, Similarly like Phase 4, I will implement versioning and create an endpoint (e.g., `http://localhost/plugin/rest/api/1.0`) to serve OpenAPI specifications for plugin REST APIs.

In Phase 6, I will Embed a mechanism within the Maven HPI plugin skeleton that automatically generates the OpenAPI specification during the initial build and on subsequent changes, including test case integration.

Note: Automated tests will be performed during each phase to enable Continuous Integration.

High-level Overview of Architecture Diagram:

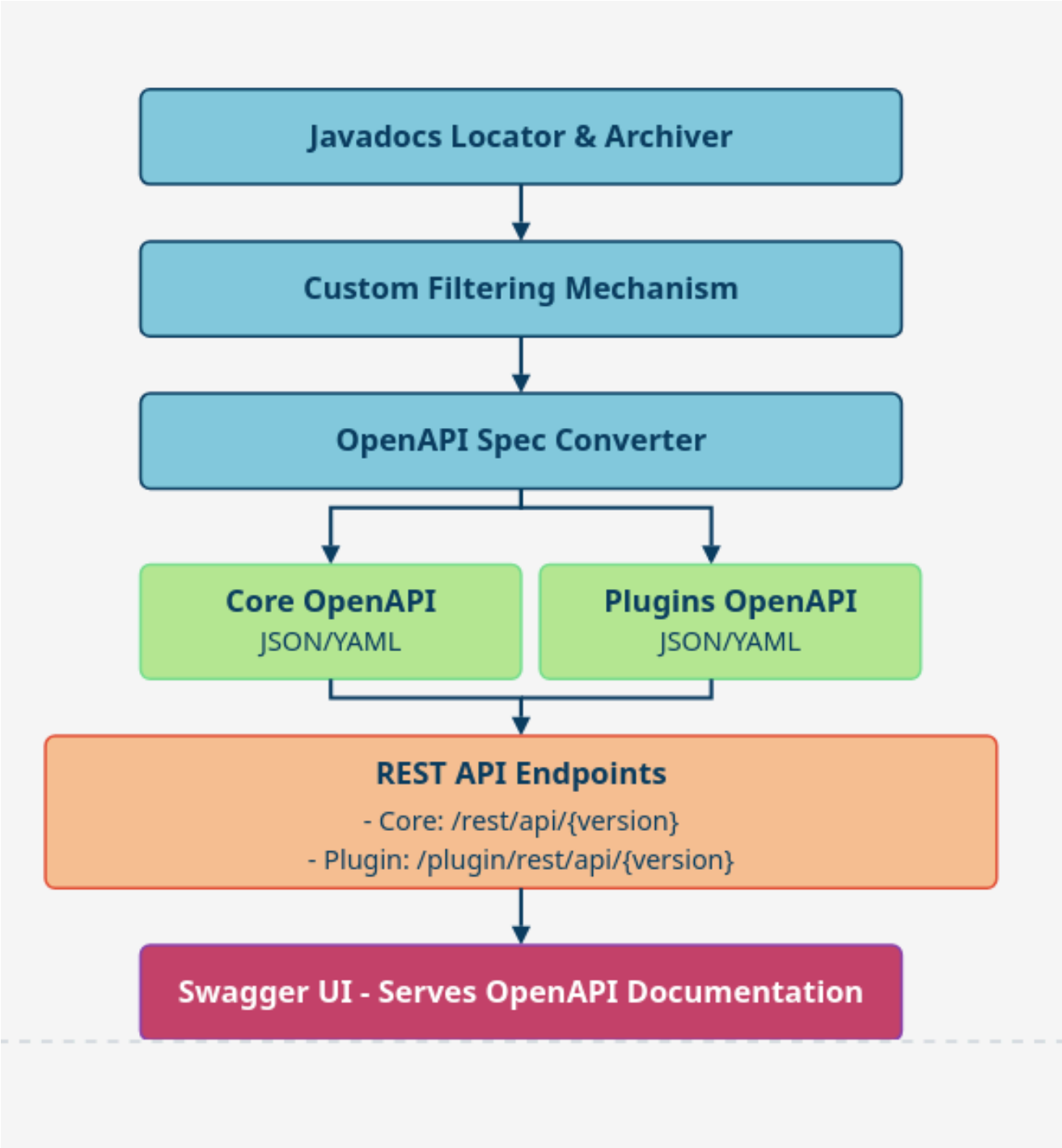


Fig 1 - Architecture diagram of new plugin

Note: Consideration & Notes for each Phase will be removed by me while submitting the final proposal & add more description in the implementation part after review.

Implementation

Phase 1: New Plugin Construction & Locate Javadocs

I will create a new plugin by following the Create Plugin Guide to set up the basic plugin structure and build configuration.

I will review the Jenkins Javadoc Plugin source code to understand how it locates the generated HTML files (typically stored in directories such as `target/site/apidocs` from Maven builds). After reviewing its code structure I will duplicate its process of locating files. All further processes of theirs will not be duplicate; rather I will add a mechanism to filter these HTML files which contain the information extracted by javadocs comments to extract only those which are essential for creating REST APIs documentation.

Considerations :

- ➔ Swagger-core can be used to extract the rest api specifications for documentations from java source code but it needs to annotate the whole code by its specified annotation. It is very time consuming to add swagger annotations to jenkins core & each plugin of jenkins.
- ➔ Java-parser can be used to scan whole source code & extract the metadata for creating the rest api documentation but it is very time consuming to add it to each plugin.
- ➔ I tried to extract rest api metadata by scanning the code for classes which contains the annotation from source code of Jenkins core but the code files of installed plugins & when new plugin gets installed are available in byte form in `war/work/plugin` directory of jenkins core in

form of `.hpi` or `.jpi`. It was difficult for me to extract from those files as I'm not much familiar with byte codes.

➔ So, I decided to create a new plugin which duplicates some code [2] from JavaDoc Plugin which contains a mechanism to work on already created HTML Documentation from javadocs comments in form of `/** ... */` in the entire codebase [3]

Note: The current idea I have proposed in this phase is based on the knowledge and resources I have preferred and gathered thus far. However, I acknowledge that there may be alternative approaches and resources that could also contribute effectively to the project. I am open to further suggestions and feedback for improvement.

Phase 2: Filtering Mechanism for REST API Extraction

I will add an extra step that parses the archived HTML files and filters out classes that expose REST API endpoints. I will do this with the help of an HTML parsing[4] library like Jsoup to read and traverse the HTML generated by the Javadoc tool.

Filtering Criteria:

- Identify classes that contain methods such as `getUrlName()` or `getApi()`.
- Look for specific annotations like `@Exported` and `@ExportedBean` that indicate REST API exposure.
- Optionally, infer the endpoint URL based on package structure or naming conventions.

Extraction:
Plugin will be able to extract relevant metadata (e.g., class names, parameters, data types, return values, method details, annotations) and temporarily store this data for later conversion.

Considerations :

➔ For generating expected http response it is very difficult & very rare possibility to get it extracted from stapler annotations because

considerations in Phase 1 and difficult to extract rest api endpoint URL & response statues for some nested classes is challenging as the path to hit the http request needs to be created by observing the hierarchy (Inherited classes from parent class) to add those in OpenAPI specifications for further creation of documentation.

➔ So, I decided to parse the extracted javadoc HTML pages to get those data for creating Open API specifications.

➔ Extracting an actual endpoint URL for some nested classes is still challenging as the path to hit the http request needs to be created by observing the hierarchy (Inherited classes from parent class) from javadocs.

Note: The current idea I have proposed in this phase is based on the knowledge and resources I have preferred and gathered thus far. However, I acknowledge that there may be alternative approaches and resources that could also contribute effectively to the project. I am open to further suggestions and feedback for improvement.

Phase 3: Conversion to OpenAPI Specification

I will map the extracted REST API details into a structured OpenAPI specifications document[5].

Mapping Data:

- Define paths, operations (HTTP methods), parameters, responses, and other OpenAPI elements based on the extracted metadata.
- Separate the conversion output into two directories: one for Jenkins core APIs and another for plugin APIs.

Libraries & Tools:

- Consider using swagger-core or similar libraries to programmatically construct the OpenAPI spec.

Storage:

Plugin will be able to store the generated spec files in a structured folder hierarchy that reflects their source (e.g., `openapi/core` and `openapi/plugins`).

Considerations :

- ➔ Stapler maps Java objects to URLs based on naming conventions rather than explicit API definitions. There's no formal REST API specification built into the framework - URLs are dynamically generated based on object structure
  Example: A class with `getItems()` becomes accessible via `/items` without explicit declaration.
- ➔ Jenkins uses `@Exported` and `@ExportedBean` annotations[1] to mark fields/methods for REST exposure these annotations don't contain API documentation metadata (unlike Spring's `@ApiOperation` or JAX-RS annotations)
- ➔ So I decided on a mechanism to convert extracted data into Open API Specifications.

Note: The current idea I have proposed in this phase is based on the knowledge and resources I have preferred and gathered thus far. However, I acknowledge that there may be alternative approaches and resources that could also contribute effectively to the project. I am open to further suggestions and feedback for improvement.

Phase 4: Versioning and Exposing Swagger UI for Jenkins Core

I will create a versioned endpoint for Jenkins core REST API documentation that can be accessed and interactively tested.

Endpoint Setup:
Use Jenkins' extension mechanisms (such as Stapler) to create a new route (e.g., `/rest/api/1.0`) that serves the generated OpenAPI JSON/YAML.

Swagger UI Integration:
Embed Swagger UI to provide an interactive documentation and testing interface.

Versioning:
Plugin will be able to implement version control in your URL (e.g., `/rest/api/1.0`) to manage updates and maintain backward compatibility.

Phase 5: Versioning and Exposing Swagger UI for Plugins

I will provide a separate, versioned endpoint for plugin-specific REST API documentation.

- **Endpoint Setup:**
  Configure a new route (e.g., `/plugin/rest/api/1.0`) using Jenkins' extension points to serve the OpenAPI spec generated for plugins.
- **Customization:**
  I may need to dynamically manage multiple plugins and ensure that each one's documentation is correctly mapped and accessible.
- **Versioning:**
  Maintain a versioning scheme similar to the core endpoint, so updates can be handled gracefully.

Considerations :

➔ For Phase 4 & Phase 5, the disparate nature of core and plugin APIs requires a cohesive management approach. So I decided to keep them in separate folders and for smooth version management.

Note: The current idea I have proposed in this phase is based on the knowledge and resources I have preferred and gathered thus far. However, I acknowledge that there may be alternative approaches and resources that could also contribute effectively to the project. I am open to further suggestions and feedback for improvement.

Phase 6: Automation Integration with Maven HPI Plugin Skeleton

I will integrate the OpenAPI generation process into the plugin build lifecycle so that the specification is automatically created the first time (and updated with changes) using Maven.

I will configure the plugin build to run the complete process (scanning, filtering, conversion, and deployment) automatically. This can be done via Maven plugins or custom goals.
I will write test cases (using JUnit or other testing frameworks) that validate each phase. Ensure that any changes to the source code or build configuration trigger tests to verify the correctness of the generated OpenAPI spec.

Proposed Timeline

Phase 0: Pre-Bonding Period (till 8th May)

- ➔ Discuss the project with mentors to brainstorm ways to improve the architecture of the tool and enhance its performance and flexibility.
- ➔ Interact with the Swagger community to refine my understanding of Swagger features and its concepts.
- ➔ Contribute to Jenkins plugins to gain more insight into the plugin environment.
- ➔ Attend meetings and office hours, and actively participate in discussions.

Phase 1: Community Bond Begins (8th May - 1st June)

- ➔ Get to know my mentors. I will establish communication through their preferred channels.
- ➔ Discuss the project in-depth with mentors and analyze better approaches to resolve the problem and I will create a final well-structured plan to implement the tool.
- ➔ I will ensure to set up the development and testing environment. Additionally, I aim to commence coding early to prevent any potential delays toward the end of the GSoC.

Phase 2: Coding Phase I (2nd June - 13th July)

The code will be committed at regular intervals after discussions and reviews with the mentors. I would prefer to build each feature based on the Test Driven Development (TDD)

I will complete implementation Phase 1, Phase 2 & Phase 3 this time & create a presentation slides for Midterm Evaluation

Note: It may take more or less near the estimated duration.

Midterm Evaluation (14th July to 18th July)

Phase 3: Coding Phase II (14th July to 25th August)

The code will be committed at regular intervals after discussions and reviews with the mentors.

I will complete implementation Phase 4, Phase 5 & Phase 6 this time.

Note: It may take more or less near the estimated duration.

Final Evaluation for Standard Coding Period (25th August to 1st September)

Major Challenges Foreseen

- Setting up a new plugin to work seamlessly with Jenkins and the Maven HPI build lifecycle can be complex. This involves understanding Jenkins' extension points, configuring Maven to run custom tasks (like scanning and archiving Javadocs), and ensuring that my plugin interacts correctly with existing mechanisms (e.g., the Jenkins Javadoc Plugin).

- Extracting REST API details from HTML Javadocs is not straightforward. I must design a filtering mechanism that accurately identifies REST API classes (using method names, annotations, etc.) among a large volume of

documentation. Variability in HTML structure between different JDK versions or custom Javadoc configurations may complicate this task.

- Mapping loosely structured metadata from the Javadocs into a strict OpenAPI (Swagger) spec can be challenging. I need to design a conversion process that correctly formats paths, methods, parameters, and responses according to the OpenAPI standard. Additionally, managing versioned endpoints for both Jenkins core and plugins adds another layer of complexity.

Availability and Commitments

The only commitments, I can commit 7-8 hours/day (including weekends) to cover 350 hours. Additionally, the code I write will not be claimed by anyone, including my university.

Continued Involvement

I am genuinely interested in plugin development. Therefore, I would be eager to adopt a deprecated plugin. Additionally, I am keen on contributing to the improvement of Jenkins UI/UX, and I intend to participate in the UX-SIG to discuss and contribute to enhancing the Jenkins user experience.

Furthermore, I will share my expertise in Jenkins with both new contributors and my colleagues at college, and I'm eager to mentor them along the way. In addition, I'm planning to establish a peer group dedicated to enhancing open-source contributions in our college through Jenkins. I believe this initiative will be valuable, especially for those who may not yet be familiar with the world of open-source contributions.

References

[1]https://www.jenkins.io/projects/gsoc/2025/project-ideas/swagger-openapi-for-jenkins-rest-api/

[2]https://github.com/jenkinsci/javadoc-plugin/blob/master/src/main/java/hudson/tasks/JavadocArchiver.java

[3]https://github.com/jenkinsci/jenkins/blob/master/core/src/main/java/hudson/FilePath.java
[4]https://jsoup.org/cookbook/extracting-data/selector-syntax
[5]https://spec.openapis.org/oas/v3.1.0.html

Relevant Background Experience

- ## With Open Source

  I am an open source enthusiast & used to be part of small open source programs for newcomers in open source. Such as -

  - GirlScript Summer of Code 24 (In top 100 contributors)
  - HacktoberFest 24 (Contributor)
  - Kharagpur Winter of Code 2024 (Contributor)
  - Social Winter of Code 25 (Contributor)
  - Innogeeks Winter of Code 25 (Contributor)

- ## With Jenkins

  I have been contributing to Jenkins since the middle of December. I have practiced using Git and GitHub and have followed the best practices. Initially, the quality of my pull requests and communication with the maintainers could have been better. I used to frequently ping the maintainers to review my PRs (which is not ideal), and my interaction was suboptimal. However, with the assistance of some maintainers, I have rectified those mistakes.

  I have explored various parts of Jenkins by contributing to documentation, UI, and plugins. Also active in discourse and open discussion for issues whenever I get stuck. Through my past contributions, I have come to understand that “Open source is not just about code; it's

also about community and collaboration" and I am grateful to be a part of this wonderful community.

Personal

I am Pratik Mane, a pre-final year undergraduate student from India. My journey into the world of Open Source began with Jenkins towards the middle of December 2024. The story of how I found my way to Jenkins still fascinates me. Our college introduced a Coursera Course in my 5th semester. In which I was attracted towards automation and came to know about Jenkins for CI/CD Pipeline.

I am a bit good at WriteUps & Researched Papers, as my interest in research with my colleagues published a Research Paper in ResearchGate and published 2 patents (1 Software & 1 Hardware) in the Indian Patent Office.

I am a quick learner and a problem solver. Aside from programming, I enjoy participating in chess tournaments, and in my free time, I often do sckething, canvas painting or spend time with my family.

Open Source Contributions

Pull Requests

1) https://github.com/jenkinsci/http-request-plugin/pull/190
2) https://github.com/jenkinsci/jenkins-clone-workspace-scm-plugin/pull/63
3) https://github.com/jenkinsci/nodejs-plugin/pull/157
4) https://github.com/jenkinsci/testng-plugin-plugin/pull/346
5) https://github.com/jenkinsci/pipeline-stage-view-plugin/pull/361
6) https://github.com/jenkins-infra/status/pull/587
7) https://github.com/jenkins-infra/jenkins.io/pull/7962

Skill Sets

Languages

- Java (4/5)
- HTML/CSS (4/5)
- JavaScript (4/5)
- Groovy (2/5)

Frameworks

- React (4/5)
- Node (4/5)
- Stapler (2/5)
- JUnit (2/5)

Tools

- Git (4/5)
- Docker (3/5)
- Linux (4/5)
- Maven (3/5)
- Gradle (2/5)
- Jenkins (3/5)
- JSON (4/5)
- YAML (3/5)
- PostgreSQL (4/5)
- MongoDB (4/5)
- Postman (4/5)
- Swagger (2/5)

Architectural Style

- REST APIs (4/5)

Reference Links and URLs

- **GitHub:** Pratik_0112

- **Mail:** pratikmane610@gmail.com
- **Portfolio:** pratik0112-portfolio
- **Element:** @pratikmane0112:gitter.im
- **LinkedIn:** Pratik Mane
- **X:** @itsPratik_0112
- **Timezone:** Indian Standard Time (IST), UST+5:30
- **Location:** Pune, India